

Raha V4 Web Dashboard: System Architecture Inventory

Last updated: 2026-03-05

1) System Overview

This project is a Next.js 16 App Router application that provides:

- Role-based dashboard (admin/editor)
- Editor Pro canvas for creating templates (text/image/video)
- Template library management (draft/published)
- Canva import (URL + Chrome extension path)
- Vector/PSD import
- Mobile-facing template APIs
- Supabase Auth + Supabase Storage integration
- PostgreSQL persistence through Prisma

Core application path: `/Users/khalidabuhayea/AndroidStudioProjects/web-dashboard`

2) Major System Components

Component	Responsibility	Main files
Dashboard shell	Authenticated app shell, left navigation, top user bar	<code>src/app/(dashboard)/layout.js</code> , <code>src/app/(dashboard)/DashboardNav.js</code>
Dashboard pages	Templates, Canva Import, Vector Import, Settings, Users, Analytics, Push, Design System	<code>src/app/(dashboard)/**/*.js</code> , <code>*Client.js</code>
Editor Pro shell	Editor route protection + shell composition	<code>src/app/(editor)/layout.js</code> , <code>src/app/(editor)/editor-pro/page.tsx</code>
Editor UI composition	Toolbar, SidePanel, Properties, Timeline, canvas stage	<code>src/components/editor/EditorLayout.tsx</code>
Canvas engine	Render/manipulate text/image/video/shapes, selection, transform, clipping to page	<code>src/components/editor/CanvasEditor.tsx</code>
Editor state store	Central state/actions/history/template meta/font registry	<code>src/store/editorStore.ts</code>
Template service	CRUD, publish/unpublish, revisions, rollback, filtering	<code>src/app/api/templates/**</code>
Import pipeline: Canva (URL)	Canva URL scrape/import path	<code>src/app/api/tools/canva-import/route.js</code>

Import pipeline: Canva extension	Tokenized extension import endpoint, layer parity metadata	<code>src/app/api/tools/canva-import/extension-token/route.js</code> , <code>src/app/api/tools/canva-import/extension-import/route.js</code>
Import pipeline: Vector/PSD	PSD/SVG/PDF processing and import	<code>src/app/api/tools/vector-import/route.js</code>
Media upload service	Upload image/video/font to Supabase Storage	<code>src/app/api/editor/media/route.js</code>
Custom fonts service	Save/list/delete custom fonts (URL/data based), app settings persistence	<code>src/app/api/editor/fonts/route.js</code> , <code>src/lib/editor/customFonts.server.js</code>
Mobile API	Signed endpoints for published templates	<code>src/app/api/mobile/templates/route.js</code> , <code>src/app/api/mobile/templates/[slug]/route.js</code> , <code>src/lib/mobile/auth.js</code>
Admin API	User management + stats	<code>src/app/api/admin/users/route.js</code> , <code>src/app/api/admin/stats/route.js</code>
Taxonomy settings	Template category/subcategory settings	<code>src/app/api/settings/template-taxonomy/route.js</code> , <code>src/lib/templates/templateSettings*.js</code>
Auth + session bridge	Supabase client/server/admin clients, session refresh proxy	<code>src/lib/supabase/*.js</code> , <code>src/proxy.js</code>
Canva extension package	Chrome extension to extract Canva page/layers and send to API	<code>extension/canva-importer/*</code>
Utility scripts	CLI Canva import + mobile signature helper	<code>scripts/import-canva-template.mjs</code> , <code>scripts/sign-mobile-request.mjs</code>

3) Databases and Storage

3.1 Physical data services

Service	Usage in this system
PostgreSQL (Supabase)	Main relational database for app tables and Supabase-managed schemas
Supabase Storage	Media object storage (uploaded image/video/font files)
Supabase Auth	User identity and JWT claims (<code>user_role</code>)

There is one primary PostgreSQL database connection configured via `DATABASE_URL` .

3.2 Application tables (public schema)

Table	Purpose	Key fields
Template	Main template records	id, ownerId, name, slug, status, version, canvasSize, category, subCategory, tags, thumbnailDataUrl, data, timestamps
TemplateRevision	Version snapshots/history	id, templateId, version, action, actorId, snapshot, createdAt
AppSetting	Key-value JSON settings	key, value, timestamps
user_roles	RBAC role mapping (admin/editor)	user_id, role (enum app_role)

3.3 Supabase-managed schemas/tables used

Schema	Notes
auth	Supabase Auth users/sessions (auth.users)
storage	Buckets/objects metadata for Supabase Storage

3.4 Media storage policy in current code

Media type	Current persistence
Images uploaded from editor	Supabase Storage URL
Videos uploaded from editor	Supabase Storage URL
Fonts uploaded from editor	Supabase Storage URL (legacy data URL still supported)
Template records	Stored in PostgreSQL Template/TemplateRevision

Default bucket for editor media: `editor-media` (override with `EDITOR_MEDIA_BUCKET`).

4) Technology Stack

4.1 Frontend and UI

Technology	Version/notes	Where used
Next.js	16.1.6 (App Router, Turbopack build)	full web app
React	19.2.3	client components
TypeScript	strict mode enabled, mixed JS/TS (<code>allowJs: true</code>)	editor + APIs + store

Tailwind CSS	v4 via @tailwindcss/postcss	styling
Lucide React	icon system	dashboard/editor UI
Zustand	editor state management	src/store/editorStore.ts
React Konva + Konva	canvas rendering/interaction	CanvasEditor.tsx
use-image	image loading hook for Konva nodes	canvas image/video rendering
shadcn-style UI primitives	local component wrappers	src/components/ui/*

4.2 Backend/API and data

Technology	Where used
Next.js Route Handlers	all /api/* endpoints
Prisma ORM	PostgreSQL app tables (Template, TemplateRevision, AppSetting)
Supabase JS + SSR	auth/session in browser/server/proxy contexts
Supabase Admin client	privileged operations and storage uploads

4.3 Import and media processing

Technology	Purpose
Playwright	Canva scraping/import automation
ag-psd	PSD parsing
node-canvas (canvas)	server-side canvas operations
pngjs	PNG parsing/processing

4.4 Tooling

Tool	Purpose
ESLint + eslint-config-next	linting
Next build pipeline	production compile/typecheck
Supabase CLI config + SQL migrations	local Supabase services + schema evolution

5) API Surface (Current Routes)

HTTP route	File	Responsibility
------------	------	----------------

GET /api/admin/stats	src/app/api/admin/stats/route.js	Admin dashboard counters
GET/POST/PATCH/DELETE /api/admin/users	src/app/api/admin/users/route.js	Admin user/role operations
GET/POST/DELETE /api/editor/fonts	src/app/api/editor/fonts/route.js	Custom font records
POST /api/editor/media	src/app/api/editor/media/route.js	Media upload to Supabase Storage
GET /api/mobile/templates	src/app/api/mobile/templates/route.js	Mobile list of published templates
GET /api/mobile/templates/:slug	src/app/api/mobile/templates/[slug]/route.js	Mobile template detail
GET/PUT /api/settings/template-taxonomy	src/app/api/settings/template-taxonomy/route.js	Category configuration
GET/POST/PATCH/DELETE /api/templates	src/app/api/templates/route.js	Template CRUD/list/publish
GET/DELETE /api/templates/:id	src/app/api/templates/[id]/route.js	Template detail/delete
GET /api/templates/:id/history	src/app/api/templates/[id]/history/route.js	Revision history
POST /api/templates/:id/rollback	src/app/api/templates/[id]/rollback/route.js	Rollback to revision
POST /api/tools/canva-import	src/app/api/tools/canva-import/route.js	Canva URL import
POST /api/tools/canva-import/extension-token	src/app/api/tools/canva-import/extension-token/route.js	Extension auth token
OPTIONS/POST /api/tools/canva-import/extension-import	src/app/api/tools/canva-import/extension-import/route.js	Extension payload import
POST /api/tools/vector-import	src/app/api/tools/vector-import/route.js	PSD/SVG/PDF import

6) Environment Variables in Use

Variable	Used by	Purpose
DATABASE_URL	Prisma	PostgreSQL connection

NEXT_PUBLIC_SUPABASE_URL	Supabase browser/server clients	Supabase project URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Supabase browser/server clients	public client key
SUPABASE_SECRET_KEY	Supabase admin client	privileged server key
SUPABASE_SERVICE_ROLE_KEY	Canva import token fallback	secret fallback in token signing helper
CANVA_IMPORT_TOKEN_SECRET	Canva extension auth	token signing/verification secret
EDITOR_MEDIA_BUCKET	media upload API	storage bucket override
ADMIN_EMAILS	admin users API	optional admin whitelist
ADMIN_EMAIL_DOMAIN	admin users API	optional admin domain allow list
MOBILE_API_KEY_ID	mobile auth	signed request key id
MOBILE_API_SIGNING_SECRET	mobile auth	HMAC secret for mobile endpoints

7) File Structure (Key Project Tree)

```
web-dashboard/
├ docs/
│   ├── canva-template-import.md
│   ├── editor-canva-video-checklist.md
│   └ system-architecture-inventory.md
├ extension/
│   └ canva-importer/
│       ├── manifest.json
│       ├── background.js
│       ├── popup.html
│       ├── popup.js
│       └ popup.css
├ prisma/
│   ├── schema.prisma
│   └ migrations/
├ scripts/
│   ├── import-canva-template.mjs
│   └ sign-mobile-request.mjs
├ src/
│   ├── app/
│   │   ├── (dashboard)/
│   │   │   ├── layout.js
│   │   │   ├── DashboardNav.js
│   │   │   └ DashboardClient.js
```

```

| | | | | templates/
| | | | | canva-import/
| | | | | vector-import/
| | | | | settings/
| | | | | users/
| | | | | analytics/
| | | | | notifications/
| | | | | └ design-system/
| | | | |
| | | | | (editor)/
| | | | | └ layout.js
| | | | | └ editor/
| | | | | └ editor-pro/
| | | | |
| | | | | api/
| | | | | └ admin/
| | | | | └ editor/
| | | | | └ mobile/
| | | | | └ settings/
| | | | | └ templates/
| | | | | └ tools/
| | | | |
| | | | | login/
| | | | | └ layout.js
| | | | | └ globals.css
| | | | |
| | | | | components/
| | | | | └ editor/
| | | | | └ EditorLayout.tsx
| | | | | └ Toolbar.tsx
| | | | | └ SidePanel.tsx
| | | | | └ CanvasEditor.tsx
| | | | | └ PropertiesPanel.tsx
| | | | | └ PagesTimeline.tsx
| | | | | └ ui/
| | | | |
| | | | | lib/
| | | | | └ auth/
| | | | | └ editor/
| | | | | └ mobile/
| | | | | └ supabase/
| | | | | └ templates/
| | | | | └ tools/
| | | | |
| | | | | store/
| | | | | └ editorStore.ts
| | | | |
| | | | | types/
| | | | | └ proxy.js
| | | | |
| | | | | └ supabase/
| | | | | └ config.toml
| | | | | └ migrations/

```

8) Architecture Notes

- The system uses App Router route groups:
 - (dashboard) for management UI
 - (editor) for editor screens

- Authorization model:
 - Supabase auth claim `user_role ( admin / editor )`
 - Role source table: `public.user_roles`
- Template content is stored as JSON in PostgreSQL (`Template.data`) and managed by the Zustand store format.
- Media binary files are decoupled from template rows and now uploaded to Supabase Storage.